RITSUMEIKAN

PBL4 G1-2

Multi-projector system
Team progress presentation

Ritsumeikan University
Development, Documentation and Management
12 November 2026

G1-2 project team

Assigned roles

- **DEVELOPMENT** — 4 people
  - **Haruho** — Technical Lead / PL
    - **Daisuke** — UML Designer & Developer
    - **Marcus** — Image Blending & Alignment Developer
    - **Hanz** — Testing & Experiments Developer
- **DOCUMENTATION** — 4 people
  - **Monika** — Documentation Lead / Doxygen
    - **Pan** — Code Commenter
    - **Sousuke** — Project Overview & Design Writer
    - **Bien** — Setup Instructions & Results Writer
- **PROJECT MANAGEMENT** — 4 people
  - **Jake** — Project Manager / PM
    - **Tarun** — HR & Workload Coordinator
    - **Watts** — Jira & File Delivery Coordinator
    - **Sariga** — Wiki & Meeting Notes Documenter

Development: team progress

Team contributions

Haruho — experiments and analysis
Daisuke — repository and UML
Marcus — cropping and blending
Hanz — testing and defects

Demonstrated build

main_alpha_blender.py + config.json

Latest reported result

Continuous landscape image across two 1280 × 800 projectors.
Haruho’s gamma 2.2 test shows a uniform overlap.

Remaining limitation

Projector tilt and height still require manual alignment.

Haruho 1:00 Jira: PG1-45

Test input: landscape (7680 × 3200)



Haruho (with slide 3) Same input for all Development tests

Development: demonstration

Input and settings

Landscape image (7680 × 3200, about 2.4 : 1)
Projector: 1280 × 800 each, overlap 213 px (17%), gamma 2.2

Output comparison



Gamma 1.0: overlap slightly dark



Gamma 2.2: uniform brightness

Demonstrated revision

Run: python3 main_alpha_blender.py
Keys: g / h = gamma down / up, f = full screen, q = quit

Haruho: experiments and analysis

My focus
Real-projector experiments and result analysis

Gamma comparison
Compared overlap brightness at gamma 1.0 and 2.2.
Gamma 1.0: overlap slightly dark.
Gamma 2.2: uniform brightness in the shown test.

Image choice
Used a wide landscape image (about 2.4:1)
to fit this two-projector setup.

Next step
Explore automatic alignment using feature matching
and a homography. Prototype evaluation is ongoing.



Earth image: poor fit

Jira: PG1-52

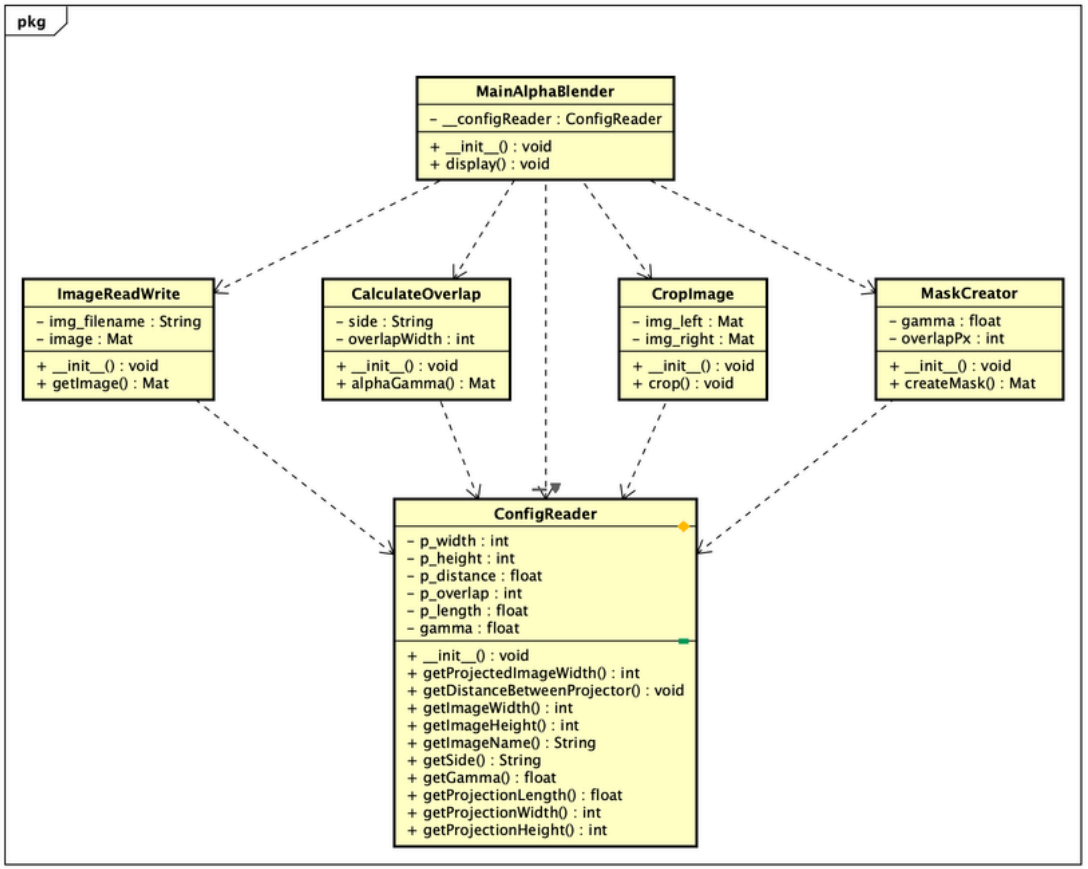
Daisuke: repository and UML

My focus
Repository structure and UML

Design decision
Reviewed ConfigReader class to store all settings in one config.json file instead of hardcoding values in each class.

Code consistency
Made sure every class has the same getter and setter methods so the code matches the UML diagram.

UML Diagram



Code: main_alpha_blender.py, config_reader.py, etc.
Diagram: 2026-11-11
Jira: PG1-53

Marcus: cropping and blending

My focus
Cropping and blending validation

Setup

Landscape input: 7680 px × 3200 px

Overlap: 213 px (about 17% of the projector width)

Method: fade across the overlap

Observed Results:
Resizing removed the grey borders.

Remaining Validation:
A dark band remained at the bottom of the overlap in this test.

Setup evidence

Same input
before and after

Each projector:
1280 px × 800 px

Gamma: 2.0

Jira: PG1-54

Before Resize



After Resize



Hanz: gamma 1.5 test

Test case
Overlap brightness at gamma 1.5

Environment
Two projectors, Python/OpenCV

Input: landscape image
Gamma: 1.5

Overlap setting: not confirmed
for this recorded test

Expected Result:
Smooth uniform brightness

Observed Result:
Visible dark band in the center

Observed Result:



Commit: 175f0b5
Test date: 5 November 2026
Jira: PG1-55

Hanz: test findings and follow-up

Recorded finding
A dark band was visible in the overlap
at gamma 1.5 on 5 November.

Follow-up result
Testing at gamma 2.2 showed improved
brightness uniformity

Remaining validation

- Confirm the original test’s overlap setting.
- Record the final configuration and repeat the brightness and alignment check on both projectors.

Documentation: team progress

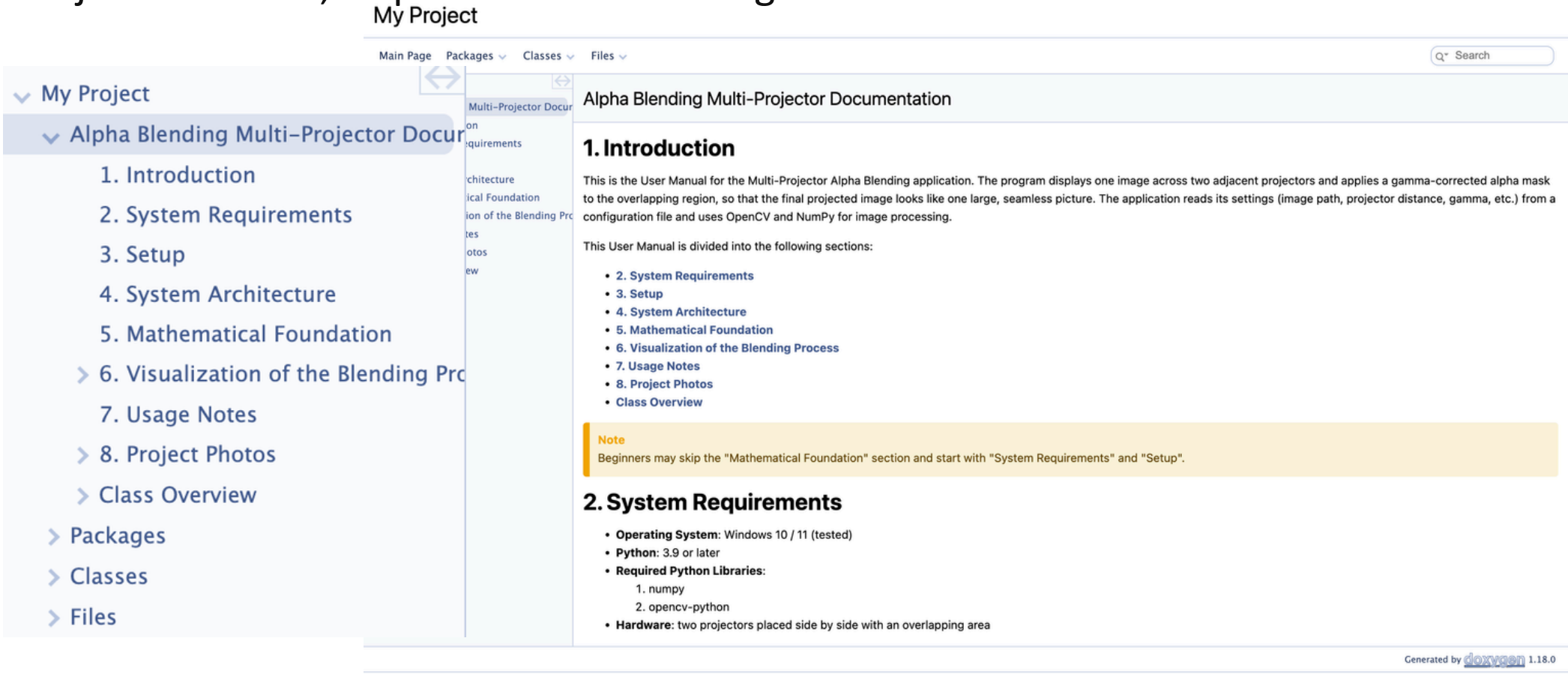
Contributions shown in this presentation
Monika — Main Page and documentation review
Pan — run instructions and source comments
Sousuke — overview and design review scope
Bien — Doxygen generation and offline files

Outputs
Main Page, detailed class pages.

Final handover checks
Confirm the documentation matches the delivered build.
Include the complete HTML folder, images and configuration.
Monika and Bien coordinate the final documentation check.

Documentation: Main Page

Project overview, requirements and navigation



Generated HTML preview

Documentation: blending explanation

Input images and blending process

6. Visualization of the Blending Process

Before Mask Application

The raw left and right projector inputs. Without the mask, the overlap region appears brighter than the rest of the image.



Left projector input



Generated HTML preview

Documentation: classes and processing flow

Class Overview

This page gives an overview of all classes in the Alpha Blender system and explains how they work together.

Click a class name to open its detailed reference page.

Class List

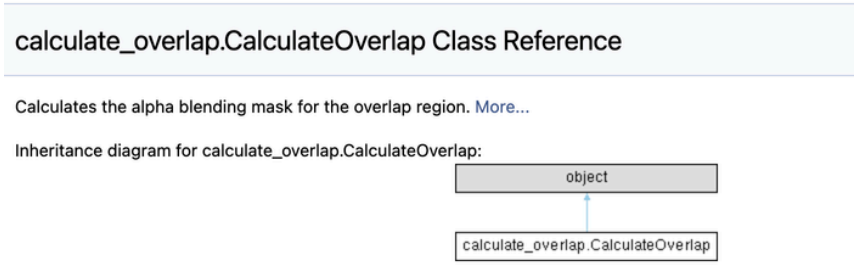
Class	File	Responsibility
MainAlphaBlender	main_alpha_blender.py	Entry point. Controls the whole pipeline and displays the final image.
ConfigReader	config_reader.py	Reads projector and image settings (width, distance, overlap, gamma, side).
ImageReadWrite	image_read_write.py	Loads the source image from disk as an OpenCV Mat.
CropImage	crop_image.py	Splits the source image into left and right parts for each projector.
CalculateOverlap	calculate_overlap.py	Calculates the overlap width and the gamma-corrected alpha values.
MaskCreator	mask_creator.py	Creates the gradient mask applied to the overlap region.

Processing Flow

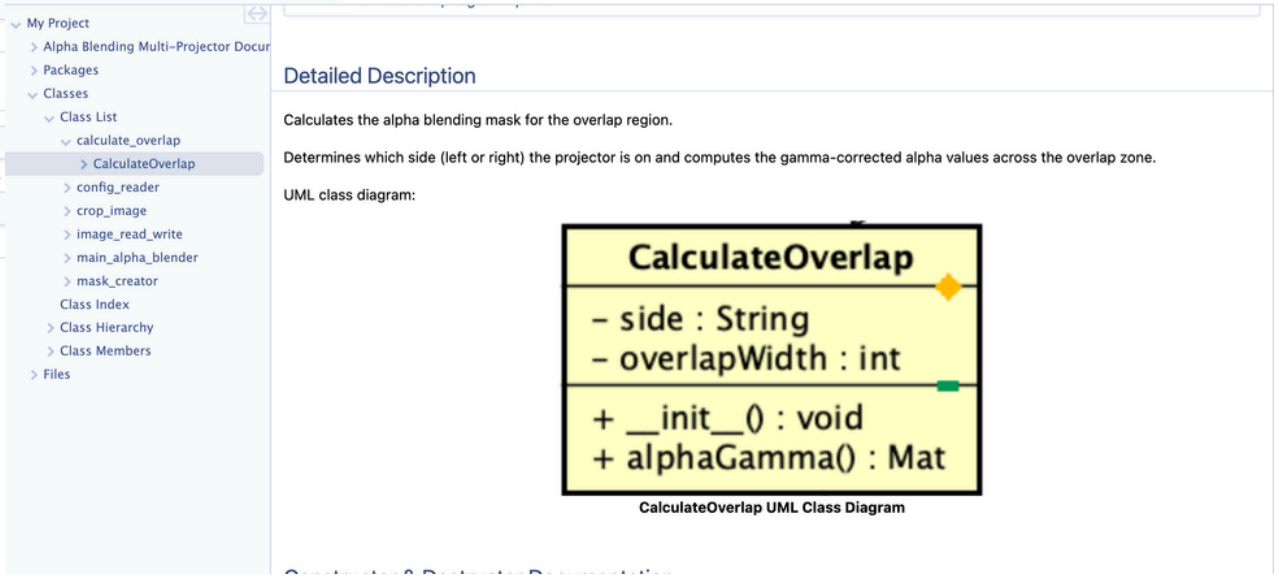
1. **MainAlphaBlender** starts the program and creates a **ConfigReader**.
2. **ConfigReader** loads all projector and image parameters.
3. **ImageReadWrite** reads the source image.
4. **CropImage** cuts the image into the left and right parts.
5. **CalculateOverlap** determines the overlap region for this projector side.
6. **MaskCreator** builds the gamma-based blending mask.
7. **MainAlphaBlender** applies the mask and displays the result in full screen.

Generated HTML preview

Documentation: class reference



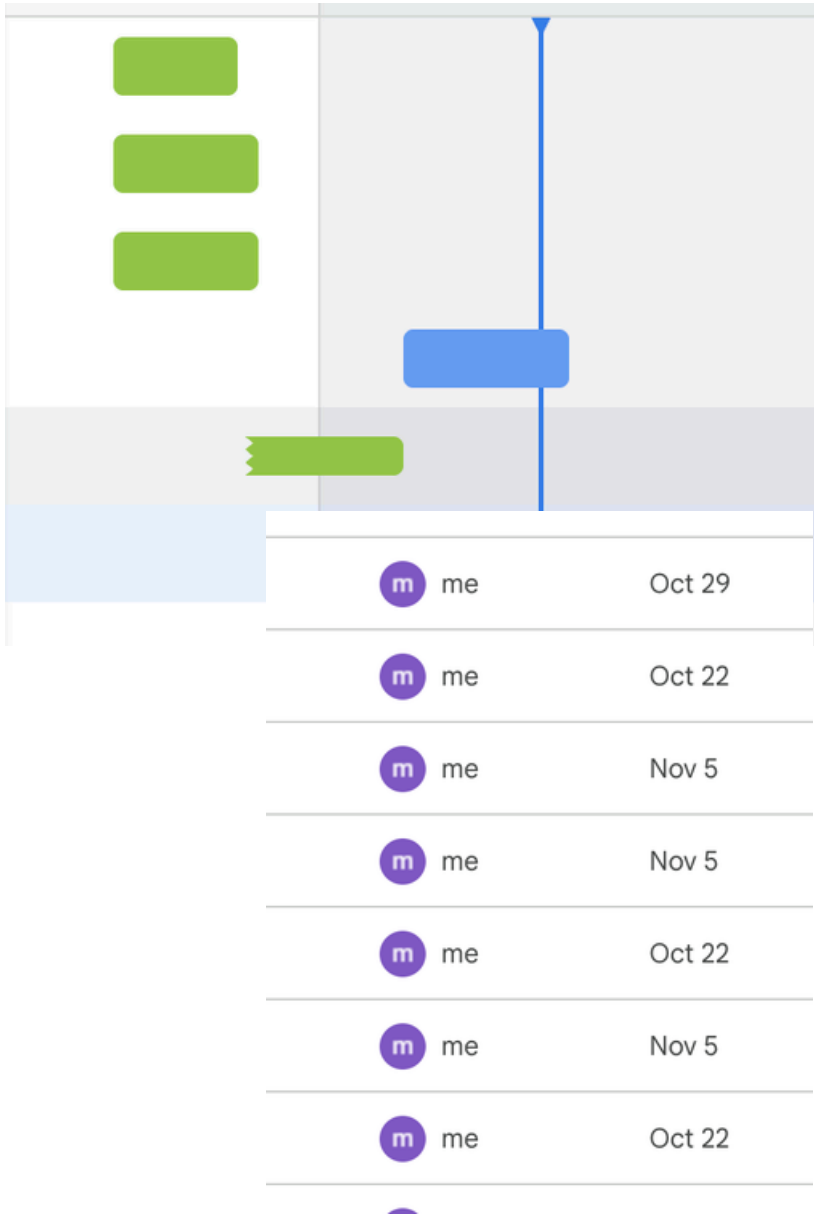
Class
methods and
diagram



Monika: documentation review

My contribution
Coordinated the documentation team.
Drafted and revised the Main Page.
Collected files and shared updates
with the other teams.
Kept Jira tasks updated.

Result
Main Page delivered with input images,
a projection photo and a team photo.
Reported mismatches to the code team.



Pan: run guide and comments

My focus
Wrote the step-by-step run instructions and clarified comments in key source files.

Reader’s task
Install the required packages, configure config.json, and start the program.

Check performed
Checked that the documented commands and configuration steps match the project files.

```
# <b>Step 1: Install required packages</b>
#
# Run the following command:
# \code{.sh}
# pip install -r requirements.txt
# \endcode
#
# <b>Step 2: Configure the settings</b>
#
# Open \c config.json and adjust the values if needed:
# - <b>image_name</b>: path to the image you want to project
# - <b>side</b>: \c left or \c right, the position of this projector
# - <b>distance_between_projectors</b>: distance between the two projectors (cm)
# - <b>projected_image_width</b>: width of one projector's image (cm)
# - <b>gamma</b>: gamma value used when creating the mask
#
# <b>Step 3: Run the program</b>
# \code{.sh}
# python main_alpha_blender.py
# \endcode
```

```
class CalculateOverlap(object):


    ## @brief Initialises CalculateOverlap with default None values.
    def __init__(self):

        ## @brief Which projector side this instance handles ('left' or 'righ
        self.side = None

        ## @brief Width of the overlap region in pixels.
        self.overlapWidth = None


    ## @brief Computes the gamma-corrected alpha blend mask.
    #
    #  Uses the formula I'(x) = (1 - x)^gamma across the overlap
    #  region to produce a smooth brightness ramp.
    #  @return Mat The computed alpha mask.
    def alphaGamma(self):
```

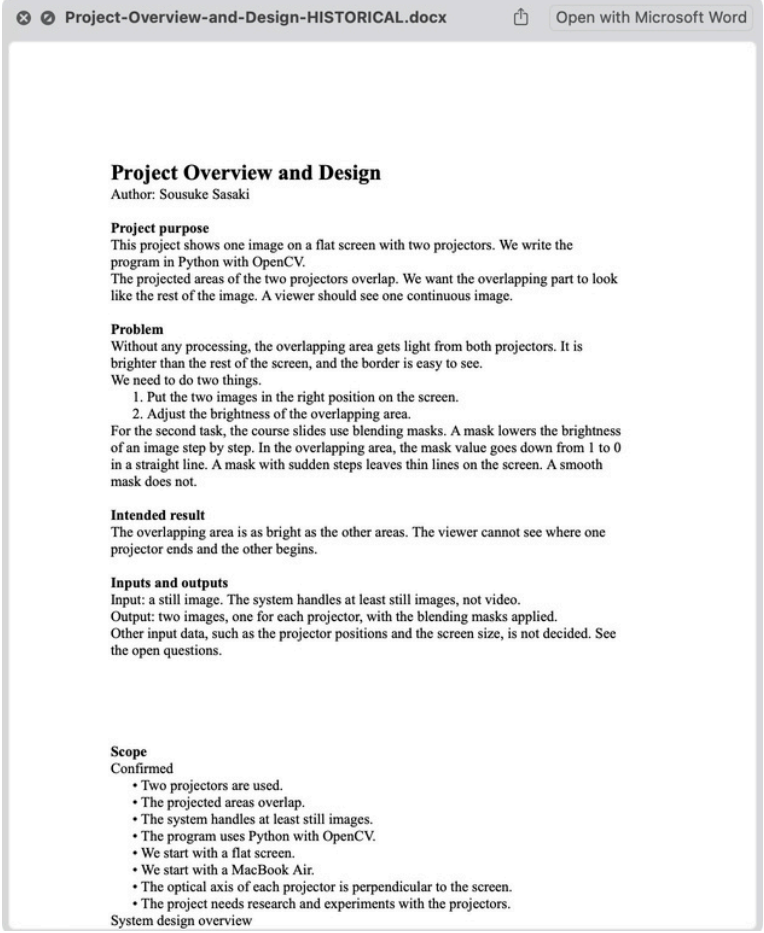
Jira: PG1-49
Pan 1:00

Sousuke (Ben): project overview

Sousuke wrote the project overview and design document.

It explains:

- The goal: one image using two projectors.
- The problem: the overlap looks too bright.
- The approach: align and blend the images.



Project-Overview-and-Design-HISTORICAL.docx — Open with Microsoft Word

Project Overview and Design

Author: Sousuke Sasaki

Project purpose

This project shows one image on a flat screen with two projectors. We write the program in Python with OpenCV.
The projected areas of the two projectors overlap. We want the overlapping part to look like the rest of the image. A viewer should see one continuous image.

Problem

Without any processing, the overlapping area gets light from both projectors. It is brighter than the rest of the screen, and the border is easy to see.
We need to do two things.

1. Put the two images in the right position on the screen.
2. Adjust the brightness of the overlapping area.

For the second task, the course slides use blending masks. A mask lowers the brightness of an image step by step. In the overlapping area, the mask value goes down from 1 to 0 in a straight line. A mask with sudden steps leaves thin lines on the screen. A smooth mask does not.

Intended result

The overlapping area is as bright as the other areas. The viewer cannot see where one projector ends and the other begins.

Inputs and outputs

Input: a still image. The system handles at least still images, not video.
Output: two images, one for each projector, with the blending masks applied.
Other input data, such as the projector positions and the screen size, is not decided. See the open questions.

Scope

Confirmed

- Two projectors are used.
- The projected areas overlap.
- The system handles at least still images.
- The program uses Python with OpenCV.
- We start with a flat screen.
- We start with a MacBook Air.
- The optical axis of each projector is perpendicular to the screen.
- The project needs research and experiments with the projectors.

System design overview

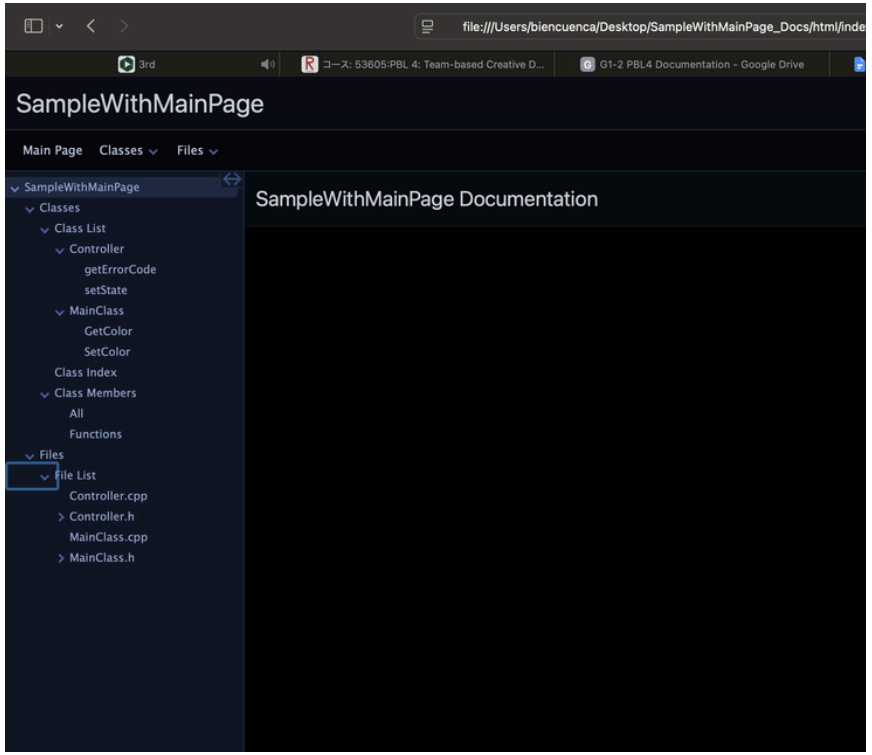
Original overview draft by Sousuke Sasaki
Presented by Ben

Bien: Doxygen and offline files

My contribution
Checked the Doxygen configuration
using Doxywizard.
Generated the HTML documentation.
Reviewed pages, images and assets.

Saved output
Doxygen configuration, HTML folder
and index.html.

Offline review
Opened the documentation locally
and checked pages and navigation.
Keep the full folder together for delivery.

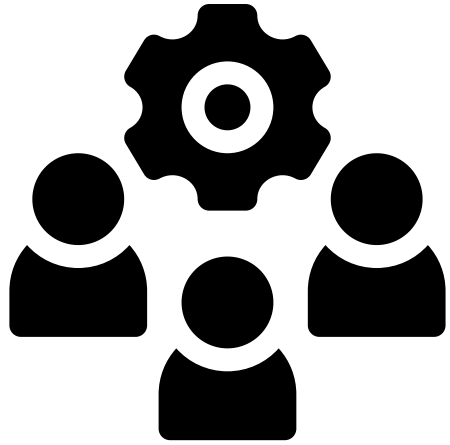


Jake: Management Overview

Our role: help the three teams work together.

- We divided the work and planned deadlines.
- We used Jira tickets and the Gantt chart to track progress of the project
- We checked in with members and collected updates.

Jake (PM): planning and team coordination
Tarun: workload and access
Sariga: wiki and meeting records
Shranjay: Jira coordination and file delivery



Jake: creating Jira tickets

Ask AI | Basic | JQL

```
project = PG1 AND creator = currentUser() ORDER BY created DESC
```

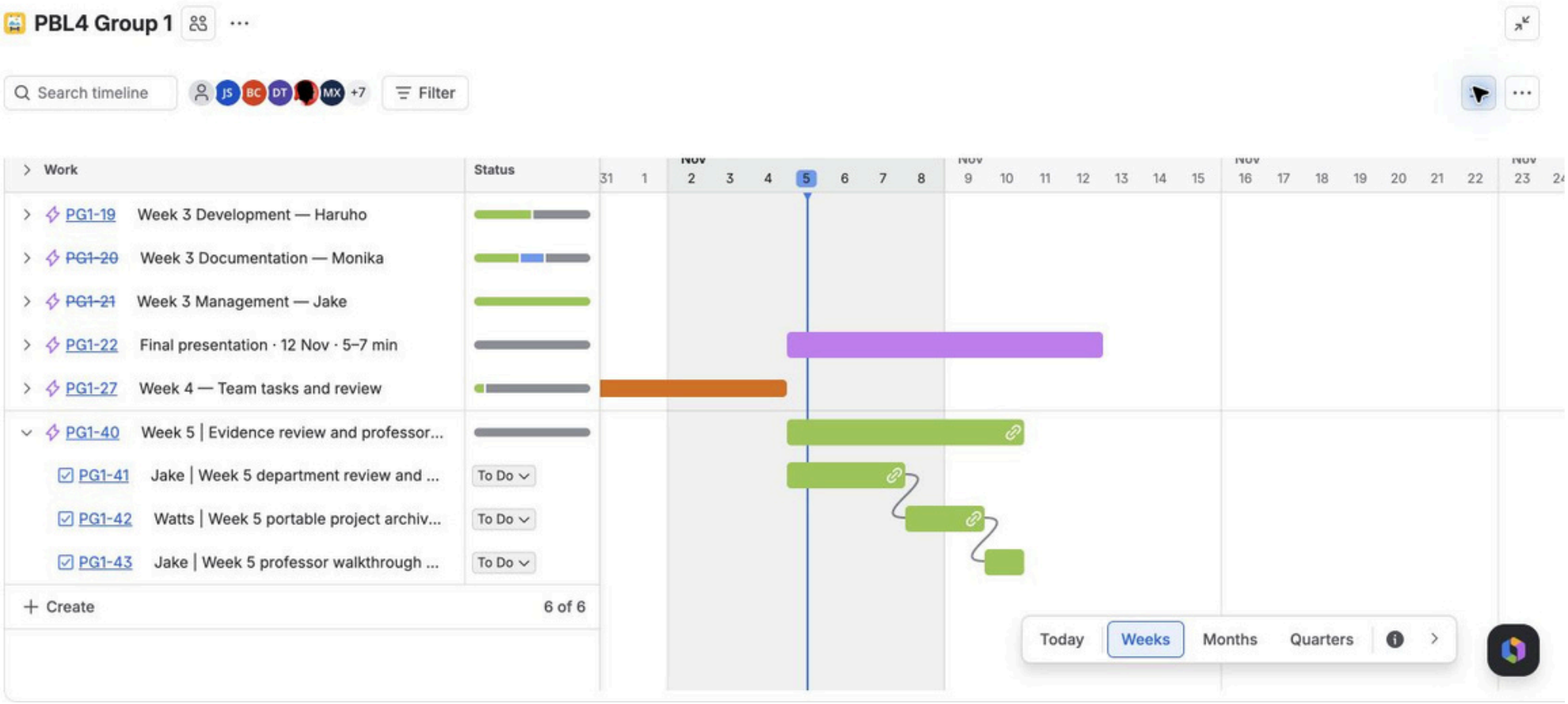
Work	Assignee	Reporter	Priority	Status
PG1-55 Hanz | Week 5 regression tests and defect register	Hanz Hankyuu	Jake Smith	Medium	To Do
PG1-54 Marcus | Week 5 cropping and blending validation...	Marcus Tiew Kai Xiang	Jake Smith	Medium	To Do
PG1-53 Daisuke | Week 5 repository and UML consistency...	Daisuke Terauchi	Jake Smith	Medium	To Do
PG1-52 Haruho | Week 5 runnable build integration and te...	山口 遥穂(is0852vv)	Jake Smith	Medium	To Do
PG1-51 Bien | Week 5 Doxygen configuration and offline a...	Bien Cuenca	Jake Smith	Medium	To Do
PG1-50 Sousuke | Week 5 overview, design and comment...	soususke sasaki	Jake Smith	Medium	To Do

I created tickets and assigned work to teammates.
I described the output needed for each task.
I split larger jobs into smaller tasks.
We used statuses to track progress.



I also helped Ben create his own Jira ticket.

Jake: Planning with the Gantt chart



I used the Gantt chart to plan tasks and deadlines.
The bars show when each task was planned.
I checked progress through meetings and individual check-ins.
This helped me see which tasks needed to happen first.

Tarun: Workload and access

My role
Check ownership, availability and access.

Recorded evidence
12-person workload table attached to PG1-10.

Tracker status · 11 Nov
PG1-10 and PG1-46: marked Done.

Remaining check
Confirm current entries and handover readiness.

PBL4 Group G1 - Week 3 Hours)

Owner: Tarun | Due Fri 23 Oct 2026 12:00 JST | Confirmed by Jake) | Individual Assignment 3 still due 27 Oct 13:00 JST

Yellow cells = still need real input from the member from the Roster sheet notes - member must confirm. Monika is assumed to be CHEN Yuanmeng (not on Roster as Monika) - please verify.

#	Name	Accepted task (proposed - confirm)	Deadline	Output & deadline understood? (Y/N)	Tool / access OK? (Y/N)	Handover OK? (Y/N)	Availability (h/week)	Assigned (h)	Load %	Blocker	Blocked by (person)	Flag
1	SMITH Jacob Daniel (Jake)	Run team meetings, agree milestones, decide redistribution I escalate	TBC	Y	Y	Y	10-Jan	9	90%	None stated	-	#REF!
2	YAMAGUCHI Haruho	Finish projector-width photo testing; integrate code into main project	TBC	Y	Y	Y	14	13	93%	Hard to find a photo wide enough to fit the projector screen	-	OK
3	TERAUCHI Daisuke	Send finished class diagrams to Documentation group	TBC	Y	Y	N	12	10	83%	None - but his handover is the dependency for 3 others	-	OK
4	MARCUS Tiew Kai Xiang	Finish image splitting / alignment / blending code	TBC	Y	Y	Y	10	8	80%	Image-splitting problem still being worked on	-	OK
5	RANEN Hanz	Next test round on Marcus output; record results; report bugs	TBC	Y	Y	Y	8	6	75%	Needs Marcus output for next tests (grayscale, resize, libs tests done)	MARCUS	BLOCKED
6	CHEN Yuanmeng (assumed = Monika)	Complete Doxygen main page; review all docs	TBC	Y	Y	Y	10	8	80%	Waiting for system + UML diagrams	DAISUKE	BLOCKED
7	PAN Lucheng	Complete Doxygen main page; code comments	TBC	Y	Y	Y	9	7	78%	Waiting for system + UML diagrams	DAISUKE	BLOCKED
8	SASAKI Sosuke	Finish last 10% of overview/design write-up (UML explanation)	TBC	Y	Y	Y	12	11	92%	Needs UML diagrams from Daisuke to finish	DAISUKE	BLOCKED
9	ALOLOD Jon Bien Cuenca	Finish setup instructions + results write-up (captions done)	TBC	Y	Y	Y	8	7	88%	Waiting for system to be completed	HARUHO / MARCUS	BLOCKED
10	KANDASWAMY Tarun	This 12-person table + workload report; chase availability	TBC	Y	Y	Y	12	11	92%	Availability not yet collected from members; need Jira keys from Watts	WATTS	BLOCKED
11	HAREESH Sariga	Wiki + meeting notes; merge wiki with Confluence	TBC	Y	Y	Y	8	7	88%	None stated (Confluence merge in progress)	-	OK
12	WATTS Shranjay	Keep Jira owners/deadlines current; prepare delivery package	TBC	Y	Y	Y	10	9	90%	None stated - supplies Jira keys for this table	-	OK

Assumption: availability and assigned hours are later (not invented). Load % = Assigned / Availability; >100% = overloaded.

Flags and proposed redistribution (for Jake to decide)

Based on the dependencies in the Roster notes. Hour-based overload cannot be confirmed until availability is collected - update after that.

	Issue	People affected	Proposed action for Jake	Needed from	By when
1	UML class diagrams not yet handed over - blocks 3 people	Daisuke -> Sosuke, Pan, Chen/Monika	Jake to confirm Daisuke sends diagrams to Documentation today; Sosuke then finishes the last 10%	Daisuke	Fri 23 Oct 12:00
2	Image splitting still unresolved - blocks testing and the final system	Marcus -> Hanz, Haruho, Bien	Move Hanz (finished his first tests) to pair with Marcus on splitting/alignment checks, as his role already includes checking output with Marcus	Jake decision; Marcus + Hanz	Fri 23 Oct 12:00
3	Projector-width photo testing is slow for one person	Haruho	Hanz to help source/test candidate photos so Haruho can focus on code integration	Jake decision; Hanz + Haruho	Fri 23 Oct 12:00
4	Documentation writers idle until system + UML are ready	Bien, Pan, Chen/Monika, Sosuke	Do not add work now; confirm they can finish within 1 day of the system being completed (handover plan)	Jake; each member	Fri 23 Oct 12:00
5	Jira keys and deadlines missing from this table	All 12	Watts to give Jira key + deadline per person so the table can be completed	Watts	Thu 22 Oct 18:00
6	Availability not entered by anyone	All 12	Each member to reply with hours/day this week; overload flags then calculate automatically	All members	Thu 22 Oct 18:00

Tarun · 1:00

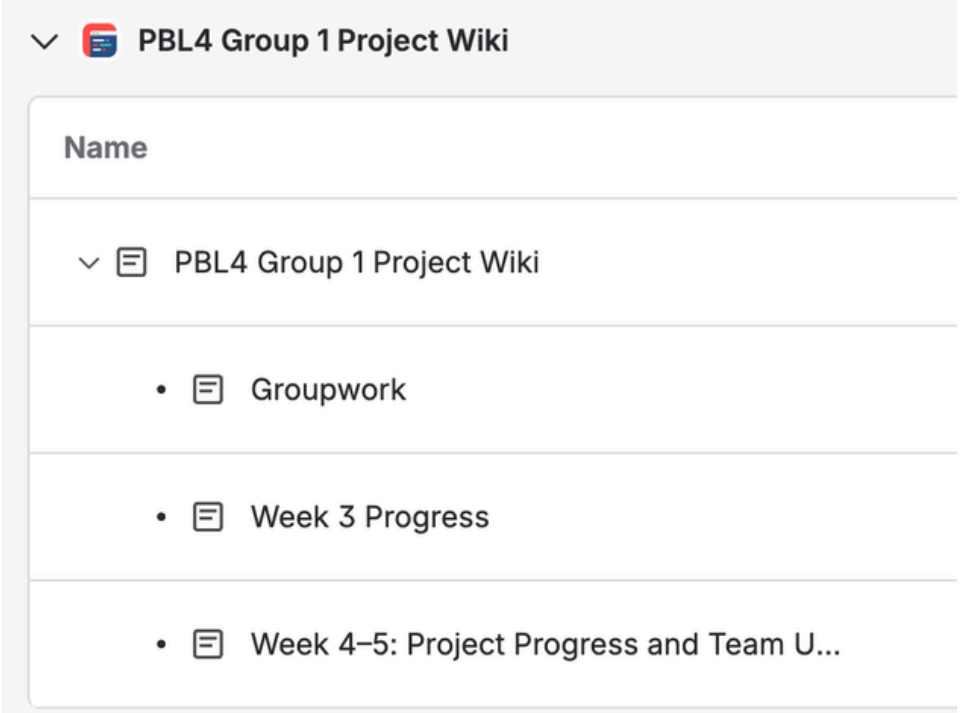
Sariga (Tarun): Meeting records

My role
Maintain the project wiki and record team progress, meeting discussions and follow-up actions.

My contribution
Created the project wiki in Confluence.
Collected development and documentation updates.
Organised responsibilities and team progress.

Recorded pages
Project Wiki · Week 3 · Weeks 4–5

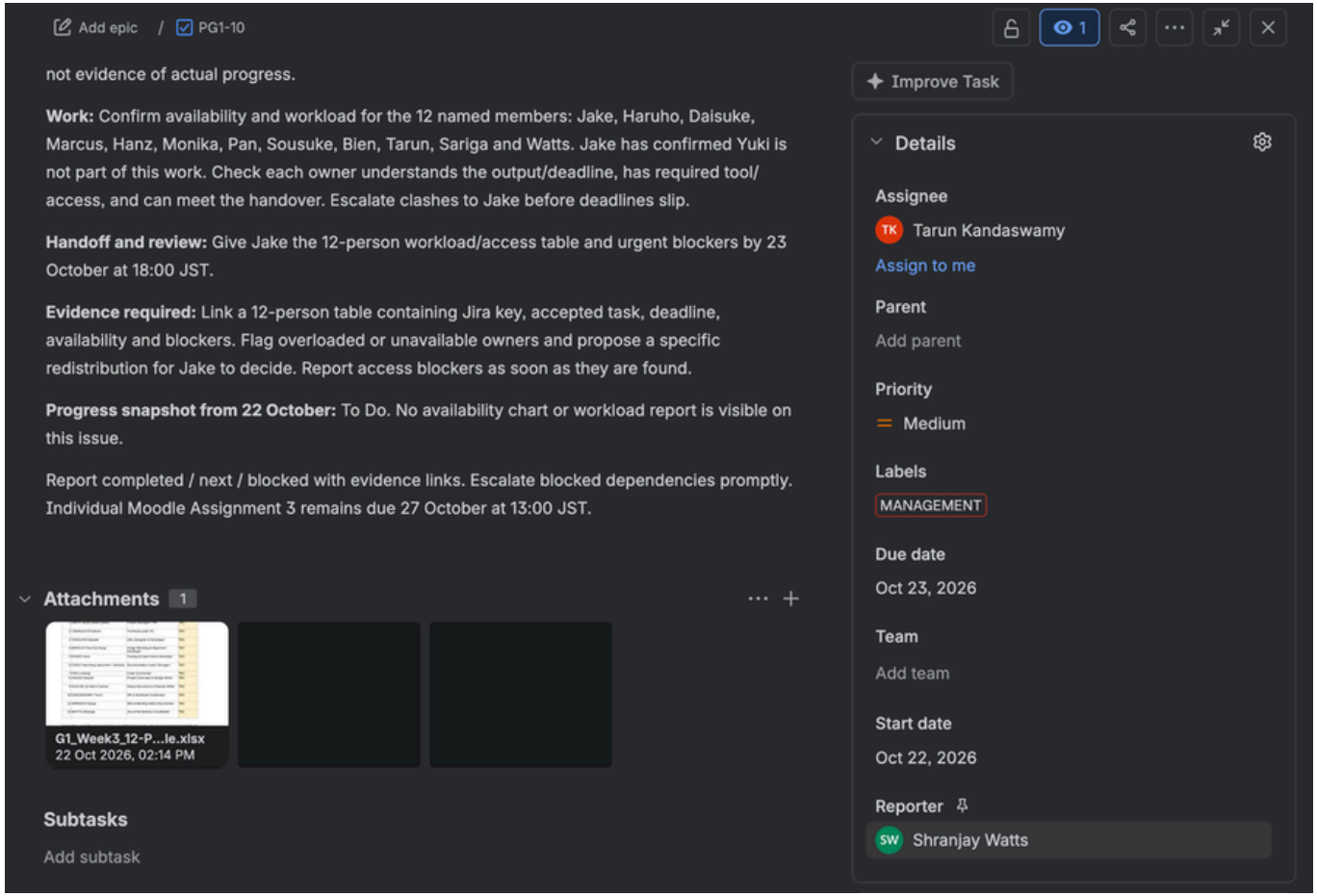
Next step
Confirm missing meeting dates, decisions and action owners.



Watts: Jira coordination

My contribution
Created and assigned team tasks.
Set deadlines and followed progress.
Created subtasks for team leaders.
Helped members use Jira.

Purpose
Make ownership and expected outputs clear across the teams.

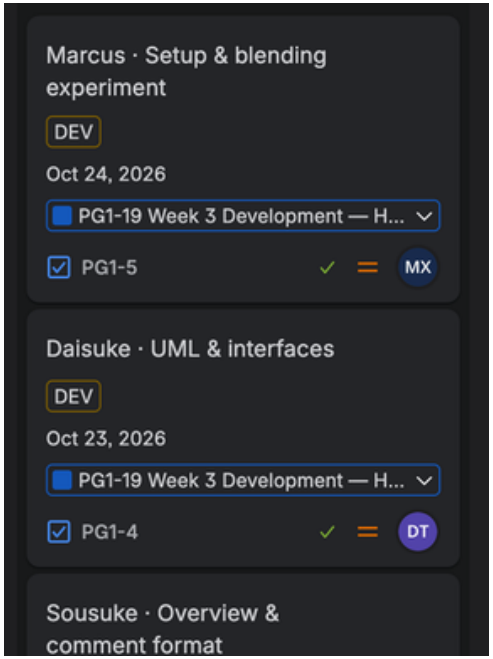


Watts 0:30

Watts: task examples and next steps

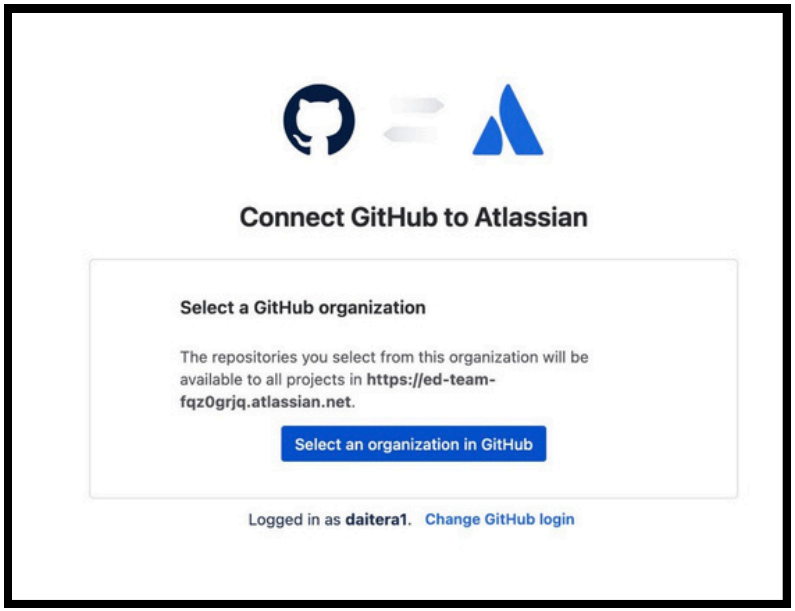
Task examples
PG1-16 — initial assignments
PG1-17 — assign Jake and Sousuke

Both marked Done in the
11 November export.



Ongoing coordination
Updated descriptions, owners and deadlines.
Tracked progress and helped teammates use Jira.

Tools explored
GitHub/GitLab integration
Confluence integration
Gantt charts for scheduling
Exploration does not mean integration is complete.



RITSUMEIKAN

Thank You!